Lecture 7. Regularization

Variable selection, ridge, lasso, elastic net, choosing the penalty

Contents

7.1 Beyond least squares **2**

7.2 Subset selection **2**

7.2.1 Choosing the optimal model 3

7.3 Regularization **3**

7.3.1 Penalized form 4

7.4 Example: delays of flights to California **5**

7.1 Beyond least squares

Motivation. With many predictors, least squares fits the noise as well as the signal and is hard to interpret. Fitting the same linear model in a smarter way, by choosing a subset of variables or by shrinking the coefficients, often predicts better and explains more simply.

The standard linear model $Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \epsilon$ extends to GLMs and to nonlinear models (splines, GAMs), yet linear models are often surprisingly competitive. Replacing least squares by other fitting procedures can improve prediction accuracy and interpretability. Three alternatives:

1. **subset selection**: use only some predictors;
2. **shrinkage** (regularization): shrink coefficients toward zero, reducing variance; some forms also select variables;
3. dimension reduction (not covered).

The same ideas apply to classification models.

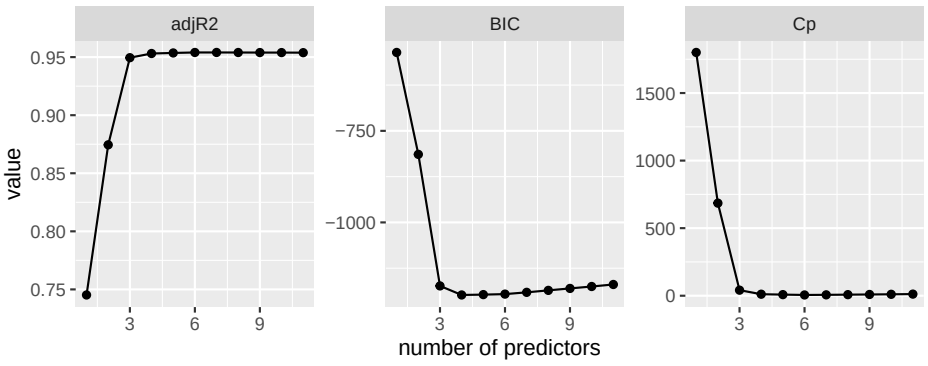
7.2 Subset selection

The fit of each candidate model is measured by its residual sum of squares $\text{RSS} = \sum_i (y_i - \hat{y}_i)^2$, the squared ($L_2$) loss.

Best subset selection fits all 2^p models: the null model M_0; for each $k = 1, \ldots, p$ all $\binom{p}{k}$ models with k predictors, keeping the best one M_k (smallest RSS); then chooses among $M_0, \ldots, M_p$. The last step cannot use RSS, which always decreases with size and would select the full model; it needs an estimate of *test* error: a validation set, C_p, BIC or adjusted R^2.

For the `ISLR2::Credit` data (credit card balance of 400 people), `leaps::regsubsets()` finds the best model of each size:

```
fit_full <- regsubsets(Balance ~ ., data = Credit, nvmax = 11)
s <- summary(fit_full)
tibble(d = 1:11, Cp = s$cp, BIC = s$bic, adjR2 = s$adjr2) %>%
  pivot_longer(-d) %>%
  ggplot(aes(d, value)) + geom_line() + geom_point() +
  facet_wrap(~ name, scales = "free_y") + labs(x = "number of predictors")
```




```
c(Cp = which.min(s$cp), BIC = which.min(s$bic), adjR2 = which.max(s$adjr2))
```

```
##    Cp   BIC adjR2
##     6     4     7
```

BIC chooses the smallest model; all three flatten out after about four predictors.

Stepwise selection. Best subset is infeasible for large p, and its huge search space also invites overfitting: the more models searched, the more likely one looks good on the training data by chance. **Forward stepwise** starts from the null model and adds, one at a time, the variable giving the largest improvement. It fits $1 + \sum_{k=0}^{p-1}(p-k) = 1 + p(p+1)/2$ models: for $p = 20$, 211 instead of $2^{20} = 1\,048\,576$, but it is not guaranteed to find the best model. **Backward stepwise** starts from the full model and removes the least useful variable at each step (requires $n > p$). **Hybrid** methods alternate: add a variable that improves the fit, then remove any that no longer help. The three approaches give similar but not identical models.

```
fwd <- regsubsets(Balance ~ ., data = Credit, nvmax = 11, method = "forward")
names(coef(fit_full, 4)); names(coef(fwd, 4))
```

```
## [1] "(Intercept)" "Income"      "Limit"       "Cards"       "StudentYes"
```

```
## [1] "(Intercept)" "Income"      "Limit"       "Rating"      "StudentYes"
```

Here the best 4-variable model (`Income`, `Limit`, `Cards`, `Student`) differs from the forward-stepwise one, which keeps `Rating` because it entered first.

7.2.1 Choosing the optimal model

Test error can be estimated *indirectly*, by adjusting the training error for the optimism due to overfitting, or *directly*, by validation or cross-validation. For a least-squares model with d predictors and $\hat\sigma^2$ the error variance estimated from the full model:

$$C_p = \frac{1}{n}(\mathrm{RSS} + 2d\hat\sigma^2), \qquad \mathrm{BIC} = \frac{1}{n}(\mathrm{RSS} + \log(n)\, d\hat\sigma^2), \qquad R^2_{\mathrm{adj}} = 1 - \frac{\mathrm{RSS}/(n-d-1)}{\mathrm{TSS}/(n-1)}.$$

- C_p adds the penalty $2d\hat\sigma^2$ because the training RSS underestimates test error. AIC, defined for any model fitted by maximum likelihood, is proportional to C_p for Gaussian linear models (up to constants).
- BIC replaces 2 by $\log n$; since $\log n > 2$ for $n > 7$, it penalises size more and selects smaller models.
- Adjusted R^2 is *maximised*; that is the same as minimising $\mathrm{RSS}/(n-d-1)$, which can rise when a useless variable is added.

Why $2d\hat\sigma^2$ [Supplement]

For least squares with d predictors (plus intercept), $\mathbb{E}[\mathrm{RSS}] = (n-d-1)\sigma^2$, while the expected squared error on new responses at the same x's is $(n+d+1)\sigma^2$. The difference is $2(d+1)\sigma^2$: the training error is optimistic by exactly twice the number of parameters times σ^2, which is what C_p adds back (the constant $2\sigma^2$ for the intercept does not affect comparisons).

Cross-validation estimates test error directly and needs no estimate of σ^2 or count of parameters, so it applies to any model.

7.3 Regularization

Motivation. Subset selection makes a yes–no decision for each variable, which is unstable and computationally hard. Regularization replaces it by a continuous penalty on the size of the coefficients, which can be optimised efficiently and tuned smoothly.

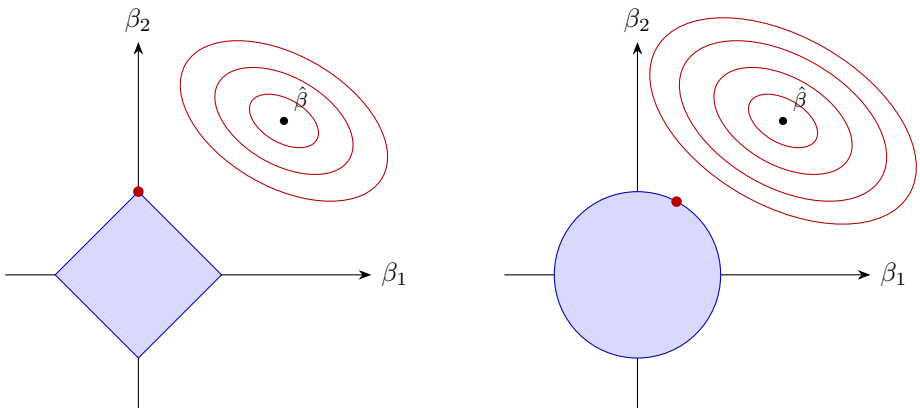
Regularization adds constraints to a model $f_\beta(x)$ to prevent overfitting. We want to minimise the average loss $\frac{1}{n}\sum_i L(y_i, f_\beta(x_i))$ and to avoid unnecessarily complex models. If complexity is the number of nonzero coefficients,

$$\hat{\beta} = \arg\min_\beta \frac{1}{n}\sum_i L(y_i, f_\beta(x_i)) \quad \text{subject to} \quad \sum_{j=1}^{p} \mathbb{1}(\beta_j \neq 0) \leq c,$$

which is best subset selection: a combinatorial, in general NP-hard problem whose constraint (L_0 "norm") is far from convex. Convex substitutes:

- the L_1 ball $\sum_j |\beta_j| \leq c_1$ (the **lasso**), which encourages sparse solutions;
- the L_2 ball $\sum_j \beta_j^2 \leq c_2$ (**ridge**), which spreads weight over features but does not give zeros;
- the **elastic net**, a compromise between the two.

Do not confuse the L_2 *loss* with the L_2 *penalty*; the lasso combines L_2 loss with L_1 penalty.



lasso: $|\beta_1| + |\beta_2| \leq c$ ridge: $\beta_1^2 + \beta_2^2 \leq c$

Figure. *Contours of the RSS (red ellipses) around the least-squares estimate $\hat{\beta}$ and the constraint regions. The constrained solution is where the smallest contour touches the region: at a corner of the diamond for the lasso (here $\beta_1 = 0$), at a generic point of the disc for ridge.* [Supplement]

7.3.1 Penalized form

With complexity $R(\beta)$, the constrained problem is equivalent to

$$\hat{\beta} = \arg\min_\beta \Big[\frac{1}{n}\sum_{i=1}^{n} L(y_i, \hat{y}_i) + \lambda R(\beta)\Big],$$

with a **regularization (tuning) parameter** $\lambda \geq 0$: ridge uses $\lambda\sum_j \beta_j^2$, the lasso $\lambda\sum_j |\beta_j|$ (the intercept is not penalised, and predictors are usually standardised first so the penalty treats them equally). The penalty is small when the coefficients are near zero, so it **shrinks** the estimates toward zero. With $\lambda = 0$ we get least squares; as $\lambda \to \infty$ the coefficients approach zero. Each λ gives a different estimate $\hat{\beta}_\lambda^R$, so choosing λ well is critical, and cross-validation is the standard tool. Shrinkage adds bias but lowers variance; for some λ the total test error is smaller than for least squares, which is why ridge and the lasso are particularly useful with many predictors.

Proposition 7.1 (orthonormal case)

Let $n = p$, $X = I$ and no intercept, so least squares minimises $\sum_j (y_j - \beta_j)^2$ with solution $\hat\beta_j = y_j$. Then ridge and lasso give

$$\hat\beta_j^{\text{ridge}} = \frac{y_j}{1+\lambda}, \qquad \hat\beta_j^{\text{lasso}} = \begin{cases} y_j - \lambda/2, & y_j > \lambda/2, \\ y_j + \lambda/2, & y_j < -\lambda/2, \\ 0, & |y_j| \le \lambda/2. \end{cases}$$

Proof [Supplement]. Both objectives are sums of one-dimensional problems. Ridge: $\frac{d}{d\beta}\{(y-\beta)^2 + \lambda\beta^2\} = -2(y-\beta) + 2\lambda\beta = 0$ gives $\beta = y/(1+\lambda)$. Lasso: $h(\beta) = (y-\beta)^2 + \lambda|\beta|$ is convex. For $\beta > 0$, $h'(\beta) = -2(y-\beta) + \lambda$ vanishes at $\beta = y - \lambda/2$, admissible iff $y > \lambda/2$; symmetrically for $\beta < 0$. If $|y| \le \lambda/2$, the one-sided derivatives at 0 are $-2y + \lambda \ge 0$ (right) and $-2y - \lambda \le 0$ (left), so 0 is the minimum. ■

Ridge shrinks every coefficient by the same proportion; the lasso shifts each toward zero by $\lambda/2$ and sets small ones exactly to zero (**soft thresholding**), which is why it performs variable selection.

7.4 Example: delays of flights to California

Arrival delays of flights from New York (EWR, JFK) to four California airports, explained by hour, origin, destination, carrier, month and day of week. The slides use `tidymodels`; the same models are fitted here with `lm()` and `glmnet()` directly.

```
California <- flights %>%
  filter(dest %in% c("LAX", "SFO", "OAK", "SJC"), !is.na(arr_delay)) %>%
  mutate(day = as.Date(time_hour),
         dow = as.character(lubridate::wday(day, label = TRUE)),
         month = as.factor(month), hour = as.factor(hour))
dim(California)
```

```
## [1] 29836    20
```

```
set.seed(386)
idx <- sample(nrow(California), size = round(0.7 * nrow(California)))
train <- California[idx, ]; test <- California[-idx, ]
flight_model <- arr_delay ~ origin + dest + hour + carrier + month + dow
mod_reg <- lm(flight_model, data = train)
tidy(mod_reg) %>% head(4)
```

```
## # A tibble: 4 x 5
##   term        estimate std.error statistic  p.value
##   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)   -16.0      6.12      -2.61 0.00905
## 2 originJFK       2.88     0.783      3.67 0.000239
## 3 destOAK        -4.61     3.10      -1.49 0.136
## 4 destSFO         1.89     0.620      3.05 0.00227
```

```
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))
rmse(test$arr_delay, predict(mod_reg, newdata = test))
```

```
## [1] 42.60868
```

`glmnet()` needs a numeric design matrix; `alpha = 1` is the lasso, `alpha = 0` ridge, and values in between the elastic net (the `mixture` argument in tidymodels):

```
X_train <- model.matrix(flight_model, data = train)[, -1]
X_test <- model.matrix(flight_model, data = test)[, -1]
mod_lasso <- glmnet(X_train, train$arr_delay, alpha = 1)
```

```
coef(mod_lasso, s = 0.01)[1:4, ]
```

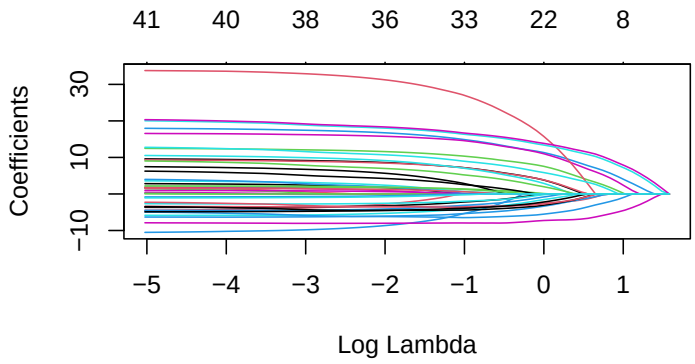
```
## (Intercept)   originJFK     destOAK     destSFO
##  -11.406671    2.783746   -4.399701    1.879023
```

```
rmse(test$arr_delay, predict(mod_lasso, newx = X_test, s = 0.01))
```

```
## [1] 42.60797
```

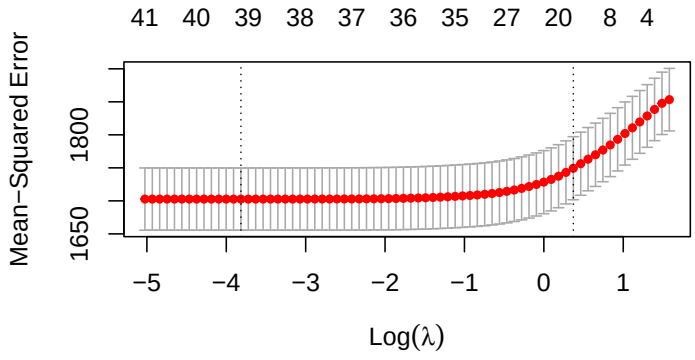
With a small penalty the lasso barely changes the coefficients (`originJFK` moves slightly toward zero) and does not improve test error here: with about 30 000 flights and 40 predictors the least-squares fit is not overfit. With many more predictors relative to n, regularization typically helps. The coefficient paths show each β_j shrinking to zero as λ grows:

```
plot(mod_lasso, xvar = "lambda")
```



Choosing λ by 10-fold cross-validation (`cv.glmnet()`), with the "one standard error" rule giving a sparser model:

```
set.seed(1)
cv_lasso <- cv.glmnet(X_train, train$arr_delay, alpha = 1)
plot(cv_lasso)
```




```
c(lambda.min = cv_lasso$lambda.min, lambda.1se = cv_lasso$lambda.1se)
```

```
## lambda.min lambda.1se
##  0.0220299  1.4494202
```

```
sum(coef(cv_lasso, s = "lambda.1se") != 0)
```

```
## [1] 19
```

```
rmse(test$arr_delay, predict(cv_lasso, newx = X_test, s = "lambda.1se"))
```

```
## [1] 43.34445
```

Ridge ($\alpha = 0$) keeps all predictors but shrinks them:

```
mod_ridge <- glmnet(X_train, train$arr_delay, alpha = 0)
coef(mod_ridge, s = 0.1)[1:4, ]
```

```
## (Intercept)   originJFK     destOAK     destSFO
##   -7.683246    2.725897   -4.448581    1.861059
```